



Tecnología informática

¿Y si el problema no es tu app, sino cómo está organizada?

Por Josué Arrese Salazar

Un servicio es una parte de la aplicación organizada para cumplir una tarea concreta. En una aplicación de comida a domicilio, una parte gestiona los pedidos, otra los pagos, otra los usuarios y otra los avisos al cliente. Así, cada función tiene su lugar y el sistema funciona de forma más clara.



uchas aplicaciones empiezan funcionando como un restaurante pequeño donde el mismo equipo se encarga de todo: tomar pedidos, cocinar, cobrar, organizar las mesas y atender las quejas. Al principio funciona, porque hay pocos clientes y todo está bajo control. Pero ¿qué pasa cuando el restaurante se llena, llegan pedidos a domicilio, se acumulan las reservas y ese mismo equipo ya no da abasto?

Cuando el todo en uno empieza a fallar

En una aplicación, esto pasa cuando toda la parte interna funciona como un solo bloque. Los programadores lo llaman **monolito**, pero la idea es sencilla: todo está unido y depende de todo.

Por ejemplo, en una app de comida a domicilio, los pedidos, los pagos y los avisos al cliente están dentro del mismo sistema. Si falla una parte, las demás también pueden verse afectadas. Es como si un problema en la caja hiciera que también se detuvieran la cocina y el reparto.

Entonces, cualquier cambio pequeño puede afectar a todo el sistema. Actualizar una notificación, modificar un pago o corregir el registro de usuarios deja de ser una tarea aislada. Todo está tan conectado que un error en una parte puede extenderse al resto. Y si una función recibe más tráfico, muchas veces no se puede reforzar solo esa zona: hay que hacer crecer toda aplicación completa.

Este problema no es solo una teoría. Empresas como Domino's Pizza UK & Ireland han tenido que modernizar sus plataformas digitales para aguantar mejor los momentos de mucha demanda y publicar cambios con más rapidez.

En negocios donde cada pedido cuenta, la forma en que está organizada la aplicación puede ser tan importante como la organización de la cocina.

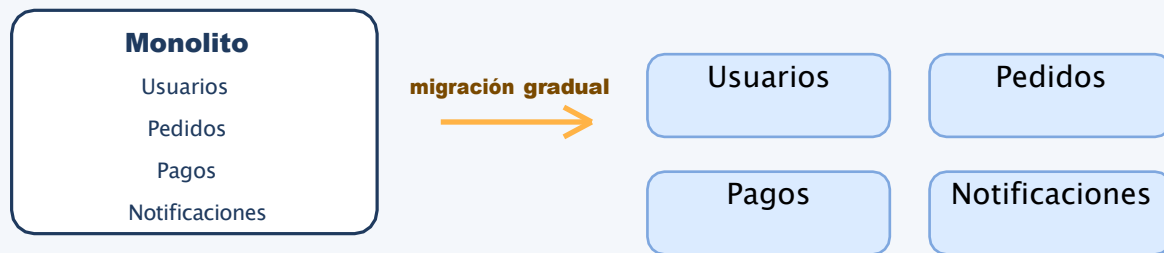
Divide y vencerás: la lógica de los microservicios

Como decía Julio César, **divide y vencerás**. Salvando las distancias, esa frase resume bastante bien la lógica de los **microservicios**: separar una aplicación enorme en piezas más pequeñas para que cada una cumpla mejor su función.

En lugar de una sola aplicación gigante, habría un servicio para usuarios, otro para pagos, otro para pedidos y otro para notificaciones. Cada uno tendría una responsabilidad clara y podría evolucionar con más autonomía.

Eso sí, la idea no es dividir por dividir. Un sistema lleno de piezas mal conectadas puede ser incluso peor que un monolito. La clave está en separar con sentido, como cuando un restaurante crece y organiza mejor sus áreas: pedidos, cocina, sala, reparto, caja y atención al cliente.

De una sola pieza a servicios con responsabilidades claras



Cada servicio puede evolucionar, desplegarse y crecer con más autonomía.

Esquema simplificado: de monolito a una arquitectura de microservicios

Una herramienta para organizar el cambio

Para hacer este proceso, **Spring Boot** sirve como punto de apoyo. Es una herramienta que ayuda a crear esas partes pequeñas de la aplicación de forma más rápida y ordenada, sin que el equipo tenga que empezar desde cero.

Además, estas partes no trabajan solas: necesitan comunicarse entre ellas. Por ejemplo, si alguien hace un pedido en una app de comida, la parte de pedidos debe avisar a la parte de pagos y, después, a la parte que informa al cliente.

Para que ese intercambio no sea un caos, suele existir una especie de **puerta principal** que recibe las peticiones y las manda al lugar correcto. Sería como el encargado del restaurante que escucha lo que necesita el cliente y lo dirige a cocina, caja o reparto.

No se reforma un restaurante en plena hora punta

Un cambio así no debería hacerse de golpe. Cambiar toda la aplicación a la vez sería como reformar un restaurante entero mientras sigue lleno de clientes. Lo más prudente es empezar poco a poco, por una parte que sea fácil de separar y que no ponga en riesgo todo el sistema.

Así, la empresa puede comprobar si esta nueva forma de trabajar realmente ayuda: si permite hacer cambios con más facilidad, evitar que un error afecte a todo y responder mejor cuando hay más usuarios. Si funciona, se puede continuar con otras partes. Si no, al menos se habrá aprendido sin tocarlo todo.

Igual que un restaurante no puede depender siempre del mismo equipo para hacerlo todo, una aplicación que crece tampoco puede apoyarse eternamente en una sola estructura. A veces, crecer no consiste en hacer más grande la cocina, sino en organizar mejor el restaurante.



Fuentes consultadas: artículo base: From Monolith to Microservices: A Spring Boot-Driven Approach to Modular Back-End Systems. Ejemplo empresarial: Rackspace Technology, caso de Domino's Pizza UK & Ireland, donde se explica la modernización de su plataforma digital hacia microservicios en Azure Kubernetes Service para mejorar la escalabilidad y el despliegue de funciones: <https://www.rackspace.com/case-studies/dominos-pizza-uk-ireland-ltd>